



EDUCACIÓN
SECRETARÍA DE EDUCACIÓN PÚBLICA



TECNOLÓGICO
NACIONAL DE MÉXICO

INSTITUTO TECNOLÓGICO SUPERIOR DE SAN PEDRO DE LAS COLONIAS



INGENIERÍA EN SISTEMAS COMPUTACIONALES

Fundamentos de Ingeniería de Software

Grupo: 5A

U1-E02 — Análisis individual de problemáticas de Ingeniería
de Software

Caso C02 — Plataforma de inscripción escolar

DOCENTE:

RUTH AIVI CHÁVEZ RODRÍGUEZ

Nombre del estudiante

Genesis Marisol Lopez Medina

Análisis del Caso B: Plataforma de Inscripción Escolar

Descripción del caso y consecuencias

El problema principal es que el sistema de inscripción solo se probó con pocas personas y funciona bien, pero cuando todos los alumnos entran al mismo tiempo en el día que existe más demanda, la página se satura y empieza a fallar: duplica solicitudes y entrega comprobantes a alumnos en materias que ya no tienen cupo.

- **Estudiantes (Usuarios):** Se estresan y se confunden porque la página va muy lenta, se les traba la inscripción y al final no saben si realmente quedaron inscritos o no porque el sistema les da comprobantes falsos.
- **Equipo de Desarrollo y Soporte:** Están saturados recibiendo quejas por todos lados sin saber dónde está la falla exactamente, porque no tienen orden en los reportes ni forma de regresar a una versión anterior que sí funcionaba.
- **La Escuela / Organización:** Queda muy mal parada frente a los alumnos y hay discusiones entre departamentos porque cada área maneja números de cupos diferentes y nadie sabe de quién es realmente la culpa.

Cinco problemas encontrados

1- El sistema vende cupos que ya no existen y duplica las inscripciones

Causa: Cuando dos o más alumnos le dan clic al mismo tiempo para inscribirse, el sistema no bloquea el cupo mientras procesa la solicitud.

Consecuencia: Estudiantes reciben un comprobante de una materia o grupo que en realidad ya está lleno.

Práctica de Ingeniería de Software: Manejo de transacciones y concurrencia en la base de datos (para que solo deje pasar una petición a la vez por cupo).

Evidencia del caso: *"algunas solicitudes se procesan dos veces y ciertos estudiantes reciben un comprobante, aunque el cupo ya se agotó."*

2- No probaron qué pasaría cuando se conecten muchos alumnos

Causa: Solo hicieron pruebas con poquitos usuarios y nunca simularon un día de inscripciones reales.

Consecuencia: La plataforma se pone lenta y termina cayéndose el día que más se necesita.

Práctica de Ingeniería de Software: Pruebas de carga y estrés (*Performance Testing*).

Evidencia del caso: *"Durante las pruebas con pocos usuarios, todas las funciones... responden correctamente" y "No se realizan pruebas de carga..."*

3- Los cambios se piden al aventón por correo o mensajes

Causa: No hay un lugar centralizado para anotar las reglas ni las decisiones que se toman sobre la plataforma.

Consecuencia: Cada departamento maneja datos distintos sobre los cupos y no se sabe qué reglas se deben programar.

Práctica de Ingeniería de Software: Gestión de requisitos y tener una "fuente única de verdad"

Evidencia del caso: *"Los cambios de requisitos llegan por correo y mensajes separados..." y "Cada área utiliza cifras diferentes..."*

4- No hay control de las versiones del código ni plan de respaldo

Causa: Programan sin llevar un registro formal de cambios ni guardar respaldos estables antes de publicar.

Consecuencia: Si la versión nueva falla el día de las inscripciones, el equipo no puede regresar rápido a una versión que sí servía.

Práctica de Ingeniería de Software: Control de versiones (como usar Git) y plan de retroceso (Rollback).

Evidencia del caso: *"...ni existe un plan documentado para regresar a una versión estable."*

5- El área de soporte recibe las quejas sin orden ni prioridad

Causa: No usan un sistema de tickets organizado ni herramientas para registrar los errores del programa en tiempo real.

Consecuencia: Soporte pierde tiempo intentando adivinar qué pasó y no sabe qué errores arreglar primero.

Práctica de Ingeniería de Software: Gestión de incidentes (Mesa de ayuda) y registro de errores (Logging).

Evidencia del caso: *"Soporte recibe reportes, pero no los clasifica por prioridad ni puede relacionarlos con una versión específica."*

3- Tres acciones prioritarias

1: Implementar control de concurrencia en la base de datos

Justificación: Es urgente evitar que el sistema entregue comprobantes falsos a los alumnos.

Costo o limitación: Tiempo para ajustar el código e interrupción breve del servicio para aplicar cambios.

Forma de comprobar la mejora: Simular 10 peticiones simultáneas al último cupo; solo una debe completarse.

2: Realizar pruebas de carga antes del periodo de inscripciones

Justificación: Asegura que el servidor soporte a todos los alumnos conectados a la vez.

Costo o limitación: Costo en herramientas de prueba y tiempo de configuración.

Forma de comprobar la mejora: Simular el tráfico máximo esperado y verificar que el sistema responda rápido sin caerse.

3: Centralizar requisitos y gestión de reportes

Justificación: Permite organizar las peticiones de cambios y priorizar los fallos de soporte.

Costo o limitación: Curva de aprendizaje del personal para usar nuevas herramientas.

Forma de comprobar la mejora: Ver que los cambios estén documentados y los reportes tengan prioridad asignada.

4. Reflexión personal

Lo que cambiaría en mi forma de desarrollar software es no asumir que por funcionar con un usuario funcionará con miles. Ahora aplicaría pruebas de carga antes del lanzamiento y mantendría un control de versiones y requisitos estricto para evitar desorganización en el equipo.

5. Fuentes consultadas y declaración de IA

Fuentes consultadas

1. Pressman, R. S. (2010). *Ingeniería del software: un enfoque práctico*. McGraw-Hill.
2. Somerville, I. (2011). *Ingeniería de software*. Pearson Educación.

Declaración sobre el uso de Inteligencia Artificial

Declaro que utilicé IA como apoyo para estructurar el documento y organizar ideas. Toda la redacción, análisis y reflexiones fueron revisadas y adaptadas por mí, cumpliendo con el límite permitido de uso de IA.